logging properties, but other state is added to the `Scope` property. Adding properties makes it easier to associate related logs with one another.

That brings us to the end of this chapter on logging. Whether you use the built-in logging providers or opt to use a third-party provider like Serilog or NLog, ASP.NET Core makes it easy to get detailed logs not only for your app code, but also for the libraries that make up your app's infrastructure, like Kestrel and EF Core. Whichever you choose, I encourage you to add more logs than you think you'll need; you'll thank me when it comes time to track down a problem.

In the next chapter we're going to be looking at your ASP.NET Core application from a different point of view. Instead of focusing on the code and logic behind your app, we're going to look at how you prepare an app for production. You'll see how to specify the URLs your application uses and how to publish an app so that it can be hosted in IIS.

Summary

- Logging is critical for quickly diagnosing errors in production apps. You should always configure logging for your application so that logs are written to a durable location such as a filesystem or other service, not just to the console, where they will be lost if the window closes or the server restarts.
- You can add logging to your own services by injecting `ILogger<T>`, where `T` is the name of the service. Alternatively, inject `ILoggerFactory` and call `CreateLogger()`.